

Proyecto trail stop

Objetivo

Un sistema con agentes que me ayude a calcular y actualizar el trailing stop de mis activos financieros. La cosa es que me gustaría saber el momento de salir sin sacrificar beneficios. Es una de las cosas que más me lastran ahora mismo.

Me gustaría hacerlo con openclaw, y delegarlo en agentes con los que me comunicaré (o con el que se suponen que sea el cerebro o el que manda) por telegram.

Me importa mucho la privacidad, por eso, me gustaría usar los agentes locales o cloud dependiendo de si la tarea requiere privacidad o no.

Tengo un minipc con un Ryzen 7 855 HS con 32 GB de RAM DDR5, no tengo GPU dedicada. Tengo ollama con estos modelos:

NAME	ID	SIZE	MODIFIED
qwen3.5:4b	2a654d98e6fb	3.4 GB	5 days ago
qwen2.5:7b	845dbda0ea48	4.7 GB	6 days ago
deepseek-r1:7b	755ced02ce7b	4.7 GB	6 days ago
nomic-embed-text:latest	0a109f422b47	274 MB	6 days ago
qwen3:8b	500a1f067a9f	5.2 GB	6 days ago

Y si algo requiere inteligencia de la buena, y capacidad de cómputo (cosa que, como ves no tengo), usar algún modelo cloud externo bueno y barato (si fuera gratis ya sería el no va más). He oído hablar bien de nemo, de qwen, de deepseek, como ejemplos de buenos para agentes y económicos.

Paso a detallar mi conversación con gemini donde recojo las especificaciones:

Yo:

Mira, como me da igual que vaya un poco lento, ¿Podría hacer el web scraping con un script en python y dejar deepseek para tareas que requieran más inteligencia que recuperar el precio de una web?

Volvamos a mi proyecto de trailing stops (que creo que es mi punto más débil en mi sistema de inversión).

Necesito organizarme cómo hacerlo. Quiero:

- tener una BD con mis tickers, donde almacene: isin y el ticker?, el exgchange?, el nombre, el sector, el precio de compra, la fecha de compra, el máximo alcanzado, el trailing stop?
- consultar para cada ticket, cada 30 minutos más o menos, su precio en ese momento (no su precio de cierre?).
- Si ese precio cumple las condiciones para ser el trailing stop (un 20% por debajo del máximo desde que compré??), actualizarlo en BD y que me avise para que yo lo cambie en trade republic.

No quiero que me lo hagas, quiero que me ayudes con las especificaciones: con eso puedo hacerlo yo a mano, o asistido por mi IA local. y también que me ayudes a plantearlo con agentes de IA: cuántos necesito, qué hace cada uno, si necesito inteligencia de verdad (el trailing stop puede calcularlo una buena IA en función de otros factores o es suficiente la fórmula que te he dicho antes del 20% del máximo desde que compré?). Qué modelo me recomiendas para cada agente, ...

También me surgen dudas sobre lo del máximo desde que compré: qué pasa si recompro acciones de un título (lo normal)? Qué pasa entonces con el máximo desde que compré (el máximo desde la primera vez que compré? desde la última? desde la más barata?)

Respuesta de Gemini:

Esta es la arquitectura de un **sistema financiero profesional y robusto**. Estás dejando atrás la idea de "un agente que navega" para construir una **máquina de gestión de activos**.

Aquí tienes la hoja de ruta técnica:

1. Diseño de la Base de Datos (SQLite)

Para gestionar recompras, lo ideal no es guardar solo "un máximo". Necesitas una estructura relacional:

- **Tabla assets:** isin, ticker, exchange, sector, nombre.
- **Tabla transactions:** id, isin_fk, fecha, precio_unitario, cantidad.
- **Tabla monitoring:** isin_fk, precio_max_historico, trailing_stop_price, ultimo_precio_leido.

Resolución a tu duda de recompras:

El **máximo histórico** debe ser el valor más alto que ha alcanzado el título desde que tienes la primera acción en cartera. Si el título sube, rompe el máximo; si recompras más barato, el máximo no cambia. **Tu trailing stop debe ser una fórmula matemática fija** (ej. $\text{\$Max} \times 0.80$) para que sea determinista. Si dejas que una IA "decida" el trailing stop, el sistema pierde su base lógica y se vuelve emocional.

2. Arquitectura de Agentes y Roles

No necesitas "Inteligencia Artificial" para calcular un porcentaje. Necesitas **Lógica Determinista** para los cálculos y **IA** para el análisis.

- **Agente 1: "The Scraper" (Script Python Puro)**
 - **Función:** Consulta Yahoo/Google Finance. **Cero IA aquí.** Solo Python + BeautifulSoup.
 - **Tareas:** Obtiene el precio actual, actualiza la tabla monitoring.

- **Modelo:** No usa IA. Es un script cron que corre cada 30 min.
- **Agente 2: "The Calculator" (Script Python Puro)**
 - **Función:** Compara precio_actual vs max_historico.
 - **Tarea:** Si $\text{Precio} < (\text{Max} \times 0.80)$, marca una bandera de "ALERTA".
 - **Modelo:** No usa IA. Lógica if/else.
- **Agente 3: "The Analyst" (IA Local - Qwen 2.5 7B)**
 - **Función:** Recibe las alertas de "The Calculator".
 - **Tarea:** Analiza el contexto. "¿Por qué ha bajado? ¿Hay noticias en los medios financieros sobre este sector?".
 - **Modelo:** Qwen 2.5 7B local (privacidad total para tus datos de cartera).
- **Agente 4: "The Messenger" (IA Cloud - DeepSeek V4-Flash)**
 - **Función:** Redactar el mensaje de Telegram.
 - **Tarea:** Convierte el informe técnico del Agente 3 en un mensaje humano, conciso y accionable para ti.

3. ¿Necesitas IA para el Trailing Stop?

Mi recomendación: No.

Usa la **fórmula del 20%** como regla estricta (determinista). La IA debe ser tu "asesor de contexto":

Cálculo: El script detecta que el precio ha caído a tu nivel de venta.

IA: "Pablo, el activo X ha tocado tu trailing stop del 20%. He revisado el mercado y parece que es una caída sectorial generalizada, no algo específico de la empresa. ¿Quieres que mantenga la orden de venta?"

4. Hoja de ruta para ti

Para no abrumarte, sigue este orden:

1. **Fase de Datos:** Crea la DB SQLite y el script de Python que hace el scrap de los 20 tickers. Asegúrate de que los datos en la tabla monitoring son correctos.
2. **Fase de Lógica:** Añade el script que calcula el 20% y detecta si se ha cruzado el umbral.
3. **Fase de Inteligencia:** Conecta el resultado de ese script a tu instancia de **OpenClaw**. Solo cuando tengas el dato puro, pide a la IA que redacte la alerta.

¿Te ayudo a redactar el esquema de las tablas SQLite para que podamos empezar por la base? Es el cimiento sobre el que todo lo demás se va a construir.

Yo:

Otra cosa, de la mayoría no me acuerdo del momento de la compra y por ende, tengo dificultades para saber su máximo... podría dar una aproximación, pero preciso no es. La primera vez, el trailing stop de todos en BD estará a 0...

Respuesta de Gemini:

Esto es un desafío muy común, pero tiene solución. Para que tu sistema sea útil desde el día uno, no necesitas el histórico perfecto, necesitas un **"punto de reinicio"**.

Si no tienes los datos históricos, no intentes reconstruir el pasado porque vas a dedicar más tiempo a buscar recibos que a invertir. Vamos a usar una estrategia de "borrón y cuenta nueva" controlada.

1. La estrategia del "Reset" (Fecha Base)

Para empezar con el pie derecho, establece una **Fecha Cero**.

- **Paso A:** Hoy (o mañana), registra el **precio de mercado actual** de todos tus activos como tu "Precio Base".
- **Paso B:** A partir de hoy, ese será tu nuevo punto de referencia para calcular el máximo.
- **Paso C:** En tu base de datos, marca una columna llamada data_quality como 'estimated' para los activos donde no tienes el histórico. Esto te permitirá, en el futuro, corregir esos datos sin que el sistema colapse.

2. Cómo manejar el "Máximo desde la compra" si no tienes el origen

Si no tienes el máximo histórico real, el trailing stop sería inútil porque el sistema no sabría desde dónde medir. **Usa el máximo de las últimas 52 semanas** (o un periodo razonable) como sustituto temporal.

- Yahoo Finance te da el "52 Week High".
- Hasta que tu sistema haya acumulado suficientes datos propios, puedes decirle al agente: *"Toma el máximo de 52 semanas de Yahoo como mi máximo inicial"*.
- Es una aproximación, pero es **infinitamente mejor que ponerlo a 0**.

3. Modificación de la Base de Datos (Estructura necesaria)

Para que esto funcione con tu Mini PC, tu tabla de monitoring debería quedar así:

tick er	precio_base_c ero	max_registra do	trailing_stop_pr ice	origen_datos
AAP L	220.00	235.00	188.00	'estimated_52w_hi gh'

- **origen_datos:** Es vital. Te ayuda a saber qué activos tienes "bien" medidos y cuáles están con aproximaciones.
- **Cuando tengas los datos reales:** Simplemente actualizas el max_registrado con el valor real y cambias el origen_datos a 'verified'.

4. ¿Cómo automatizar esto?

Puedes pedirle a tu "Agente Analista" (el que corre localmente) que haga una **tarea de inicialización**:

1. El agente consulta el precio actual y el máximo de 52 semanas de cada uno de tus 20 tickers.
2. El agente te presenta una tabla: *"Para AAPL, he tomado el máximo de 52 semanas (\$235) para fijar tu trailing inicial en \$188. ¿Estás de acuerdo?"*.
3. Tú respondes: *"Sí, aprobado"*.
4. El agente escribe esos valores en tu monitoring.db.

¿Te parece bien si empezamos por definir el script que inicializa estos valores?

Podemos hacer que primero te pida los tickers, busque él mismo los máximos de 52 semanas en la web (usando la skill de navegación o la API) y te los proponga para que solo tengas que dar el OK. Así ya tienes tu sistema de seguimiento activo sin haber tenido que buscar ni una sola factura de compra.